


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, recall_score
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from imblearn.over_sampling import SMOTE
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
df = credit_data.frame
print(f"data shape (Rows, Columns): {df.shape}")
print("Download the datasets from internet")
```

```
    ... Download the datasets from internet
```

```
df.head()
```

```
'''
   V1    V2    V3    V4    V5    V6    V7    V8    V9    V10  ...  V21    V22    V23    V24    V25    V26    V27    V28  Amount  Class
0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599  0.098698  0.363787  0.090794  ... -0.018307  0.277838 -0.110474  0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62  0
1  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803  0.085102 -0.255425 -0.166974  ...  0.225775 -0.638672  0.101288 -0.339846  0.167170  0.125895 -0.008903  0.014724   2.69  0
2 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461  0.247676 -1.514654  0.207643  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66  0
3 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609  0.377436 -1.387024 -0.054952  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50  0
4 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941 -0.270533  0.817739  0.753074  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010  0.502292  0.219422  0.215153   69.99  0
'''
5 rows x 30 columns
```

```
df['Class'] = df['Class'].astype(int)
print("Class distribution in data:\n", df['Class'].value_counts())
```

```
Class distribution in data:
Class
0    284315
1      492
Name: count, dtype: int64
```

```
X = df.drop('Class', axis=1)
y = df['Class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
print("\n --- SMOTE ... --- ")
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)
print(" after SMOT size of data:", pd.Series(y_train_smote).value_counts())
```

```
--- SMOTE ... ---
after SMOT size of data: Class
0    227451
1    227451
Name: count, dtype: int64
```

```
real_model_names = []
real_accuracy_scores = []
real_recall_scores = []
```

```
ml_models = {
'Random Forest': RandomForestClassifier(n_estimators=10, random_state=42, n_jobs =-
1),
'Logistic Regression': LogisticRegression(max_iter=500, random_state=42),
'Decision Tree': DecisionTreeClassifier(random_state=42),
'SVM': LinearSVC(random_state=42, dual=False, max_iter=2000),
'Naive Bayes': GaussianNB()
}
```

```
print("\n --- 3. Training of machine learnings models ... ---
")
```

```
for name, model in ml_models.items():
    print(f" Training: {name} ... ")
    model.fit(X_train_smote, y_train_smote)
    y_pred = model.predict(X_test_scaled)

    real_model_names.append(name)
    real_accuracy_scores.append(accuracy_score(y_test, y_pred) * 100)
    real_recall_scores.append(recall_score(y_test, y_pred) * 100)
```

```
--- 3. Training of machine learnings models ... ---
Training: Random Forest ...
Training: Logistic Regression ...
Training: Decision Tree ...
Training: SVM ...
Training: Naive Bayes ...
```

```
print("\n --- 4. (Neural Networ) Training ... --- ")
```

```

ann_real = Sequential([
    Dense(32, activation='relu', input_shape=(X_train_smote.shape[1],)),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
ann_real.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
ann_real.fit(X_train_smote, y_train_smote, epochs=5, batch_size=128, verbose=0)

ann_prob = ann_real.predict(X_test_scaled).flatten()
y_pred_ann = np.where(ann_prob > 0.5, 1, 0)

real_model_names.append('Neural Network')
real_accuracy_scores.append(accuracy_score(y_test, y_pred_ann) * 100)
real_recall_scores.append(recall_score(y_test, y_pred_ann) * 100)

print("\n --- models graph ... --- ")
--- 4. (Neural Network) Training ... ---
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1781/1781 2s 1ms/step

--- models graph ... ---

df_real_compare = pd.DataFrame({
    'Model': real_model_names * 2,
    'Percentage (%)': real_accuracy_scores + real_recall_scores,
    'Metric': ['Accuracy'] * len(real_model_names) + ['Recall (Fraud Rate)'] *
len(real_model_names)
})
plt.figure(figsize=(14, 7))
ax = sns.barplot(data=df_real_compare, x='Model', y='Percentage (%)', hue='Metric',
palette='Set2')

for p in ax.patches:
    if p.get_height() > 0:
        ax.annotate(f"{p.get_height():1f}%",
            (p.get_x()+p.get_width()/2., p.get_height()),
            ha='center', va='center',
            xytext=(0, 8),
            textcoords='offset points',
            fontsize=10, fontweight='bold')

plt.title('Final performance', fontsize=16, fontweight='bold')
plt.xlabel('ML and DL Algo', fontsize=12)
plt.ylabel('percentage (%)', fontsize=12)
plt.ylim(0, 115)
plt.legend(title='Metrics')
plt.tight_layout()
plt.show()

```